

Aplicações e Serviços de Computação em Nuvem

Trabalho Prático

Instalação, Configuração, Exploração e Otimização da Aplicação Laravel.io

Grupo 1

Hugo Martins
A95125



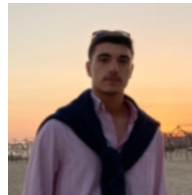
Afonso Bessa
PG53597



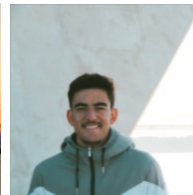
Francisco Claudino
PG50380



Ivo Ribeiro
PG53886



João Escudeiro
A96075



29 de Dezembro de 2023



Universidade do Minho

Índice

Lista de Figuras

1	Arquitetura da Aplicação	5
2	Variação da utilização do CPU	12
3	Variação da percentagem de CPU e da Memória com Autoscaler	13
4	Variação da Requisição de Pods e Utilização de memória com Autoscaler	13

Lista de Tabelas

1	Tabela que relaciona o N° de Clientes com o Throughput Médio, para o Workflow Simples	10
2	Tabela que relaciona o N° de Clientes com o Throughput Médio, para o Workflow Normal	10
3	Tabela que relaciona o N° de Clientes com o Throughput Médio, para o Workflow Complexo	11

Introdução

No âmbito da Unidade Curricular de **Aplicações e Serviços de Computação em Nuvem** foi-nos proposto a realização de um projeto com o objetivo de aplicar os conhecimentos adquiridos na automatização dos processos de instalação, configuração, monitorização, avaliação e otimização da aplicação **Laravel.io**. Este relatório tem como finalidade esclarecer, explicar e apresentar de forma organizada todas as etapas da execução deste Trabalho Prático.

Durante a execução deste trabalho, recorreremos à ferramenta **Docker**, reconhecida como a principal plataforma de gestão de *containers*, para a criação da imagem **Docker** e do respetivo *container*.

Na segunda tarefa, utilizamos a ferramenta **Ansible** para automatizar a instalação e configuração da aplicação **Laravel.io** no serviço **Google Kubernetes Engine** (GKE) da Google Cloud. Para a realização da **Monitorização**, recorreremos a ferramentas disponibilizadas pela **Google Cloud Platform** (GCP). No que diz respeito à **Avaliação Experimental**, o seu propósito é demonstrar o método utilizado para validar experimentalmente a instalação e configuração da aplicação **Laravel.io**.

Com o objetivo de assegurar escalabilidade e resiliência, implementamos na aplicação mecanismos de replicação que contribuíram para a melhoria do desempenho e da resiliência da instalação.

Implementação da Aplicação Laravel.io

A primeira tarefa deste projeto consistiu na **Implementação e Configuração Automática** da Aplicação **Laravel.io**, que pode ser dividida em duas etapas.

A primeira etapa prende-se com a elaboração da **Dockerfile**, um ficheiro imutável que contém o código fonte, bibliotecas e outros ficheiros para que a aplicação **Laravel.io** possa ser implementada. Este ficheiro gera uma **Docker Image** que será utilizada para correr numa instância de um *container* que providenciará o serviço da aplicação **Laravel.io**.

A elaboração deste ficheiro contém vários passos, de salientar:

1. Configuração das variáveis de ambiente, neste caso o nome da Base de Dados e o Nome e a Palavra-Passe de um utilizador da aplicação, através de um *script* de inicialização.
2. Instalação de pacotes necessários ao funcionamento da aplicação, incluindo *PHP 8.2*, *Node.js*, *npm*, *servidor MySQL*, *Git*, *Composer*, entre outros.
3. Cópia o repositório do **Laravel.io** e definição da diretoria de trabalho.
4. Instalação dependências do *Node.js* e compilação da aplicação usando *npm*.
5. Instalação dependências do *Composer*, copia o arquivo de ambiente e geração uma chave de aplicação.
6. Efetua a cópia do *script* de inicialização, definição de permissões de execução, expõe a porta 8000 como a porta de acesso a aplicação a nível interno (entre *containers*) e define o ponto de entrada como o *script* fornecido.

Importa salientar que, durante a execução do *script*, são realizadas verificações nas variáveis *migrate* e *seed_database*. Se alguma destas variáveis estiver definida como *true*, procede-se à migração de dados para a aplicação e ao preenchimento da base de dados da mesma, conforme apropriado.

Após a produção da **Docker Image**, a segunda etapa deste projeto prende-se com a implementação da aplicação **Laravel.io**. Para esta parte do projeto, recorreremos às ferramentas **Ansible** e **Google Kubernetes Engine (GKE)**, tal como mencionado anteriormente.

A ferramenta **Ansible** permitiu-nos automatizar o processo de comunicação com a ferramenta **GKE**, facilitando, assim, o processo de *deployment* das diferentes partes da aplicação **Laravel.io**. Conseguimos reconhecer que, sem o uso da mesma, este processo de instalação seria muito mais difícil, trabalhoso e menos automatizado. Uma vez que, esta ferramenta disponibiliza módulos que permitem trabalhar com a plataforma **Google Cloud** e com o seu serviço de **Kubernetes**.

O serviço **GKE da Google Cloud Platform (GCP)** permitiu-nos criar diversos *containers* a correr em diferentes *pods* geridos por este serviço, com o propósito de hospedar a aplicação **Laravel.io** e os vários componentes necessários para o seu funcionamento. A **GCP** também nos disponibilizou mecanismos de monitorização que foram utilizados em fases subsequentes.

Para a instalação e configuração da aplicação **Laravel.io**, optámos pela criação de distintos *pods* de *deployment* e serviço. Nos *pods* de *deployment* encontram-se os *containers* necessários para o funcionamento do nosso programa. Para obtermos estes *containers*, recorreremos às **Docker Images** do **MySQL** e à imagem desenvolvida na primeira fase deste projeto.

Arquitetura e Principais Componentes da Aplicação

Ao analisar a arquitetura da aplicação **Laravel.io**, é possível observar que se trata de uma aplicação multi-camada, adotando a arquitetura **Cliente-Servidor** e incluindo como componentes a aplicação **Laravel.io** e uma Base de Dados.

Como acabámos de mencionar, a arquitetura da aplicação recai sobre uma arquitetura Cliente-Servidor, sendo que neste caso o servidor equivale ao *Cluster* da **Google Kubernetes Engine** (GKE). Dentro deste *cluster* estão separados a aplicação **Laravel.io** e a Base de Dados, implementados em diferentes *Pods* e a correr em múltiplos *containers*, o que faz aumentar, desta forma, a possibilidade de escalabilidade e resiliência da aplicação, já que desta forma é possível escalar independentemente cada um dos componentes, de acordo com as necessidades de cada um. Além disso, garante uma maior flexibilidade, isto é, faz com que seja possível desenvolver e/ou alterar funcionalidades de forma independente, uma vez que se trabalha apenas com o respetivo *pod*, em vez de refazer totalmente a aplicação.

Como principais componentes desta a aplicação temos:

- o *pod* com o *container* que correm a aplicação **Laravel.io**
- o *pod* com o *container* que correm a sua Base de Dados **MySQL**, ambos instalados e configurados automaticamente utilizando a ferramenta **Ansible** no serviço **Google Kubernetes Engine** (GKE), como mencionado previamente.

Na imagem a seguir podemos observar uma representação visual da Arquitetura e dos Principais componentes da Aplicação.

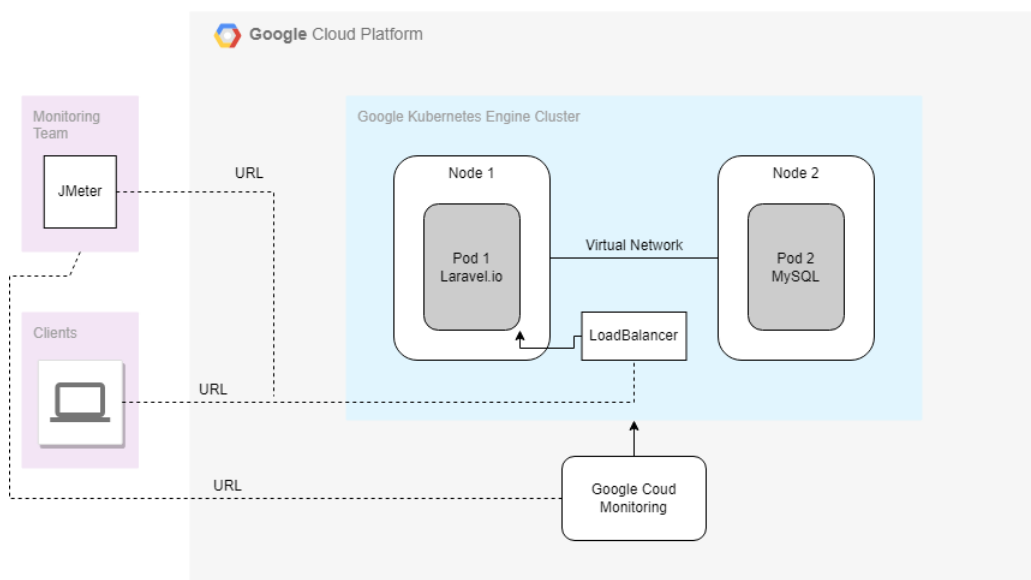


Figura 1: Arquitetura da Aplicação

Ferramentas e Abordagem Utilizadas para a Instalação e Configuração Automática da Aplicação

Após a construção da **Docker Image** do **Laravel.io**, realizada na primeira fase do projeto, tínhamos conhecimento de que existia um conjunto de variáveis de ambiente a serem introduzidas (uma vez que estas foram as variáveis configuradas no ficheiro Dockerfile). Para que a aplicação pudesse funcionar corretamente, foi necessário introduzir as seguintes variáveis de ambiente:

1. **URL**
2. **Database** (Nome da Base de Dados e o Nome e a Palavra-Passe de um Utilizador da Aplicação)

Assim, antes da implementação do **Laravel.io**, foi necessário criar um *container* com a **Image MySQL**, ficando este disponível num dado *pod*. Associado a este *pod*, definimos também um serviço que permite que a *app* comunique com a Base de Dados. Adicionalmente, de modo a que os dados armazenados na Base de Dados sejam persistentes e não se encontrem armazenados apenas durante o tempo de vida do *pod* (dados efémeros), foi necessário disponibilizar um **Persistent Volume** ao *pod* da Base de Dados. Para a criação deste, definimos um **Storage Class**. Por sua vez, este permite que o GKE crie um Persistent Volume conforme a necessidade deste para tal.

Juntamente com o *deployment* do **Laravel.io**, criámos também um **Service**. Este permitiu que a aplicação pudesse ser acedida através de um *browser* por clientes a partir do exterior. Para tal, utilizámos um **LoadBalancer** que foi pertinente para a posterior implementação da replicação. Para fornecer a variável de ambiente URL, determinámos o IP do Load Balancer definido anteriormente, e definimos que a variável URL seria composta por esse IP associado a uma porta escolhida (neste caso a porta 8000), no formato `IP:PORTA`. Este URL está sujeito a variações pois como o mesmo está associado a um LoadBalancer, quando este é reiniciado, o seu ip altera-se.

No final, conseguimos fornecer uma solução de implementação automática da aplicação **Laravel.io**. A nossa implementação segue a estrutura **Multitier**. Associamos a nossa solução a esta arquitetura uma vez que cada **Server** (Base de Dados e Servidor aplicativo), funciona como **Client** do servidor vizinho. Ao seguirmos uma arquitetura distribuída, podemos garantir que o serviço disponibilizado é escalável, mais resistente e modular.

Monitorização

Na segunda tarefa deste projeto, é pretendida a utilização de ferramentas de monitorização da **Google Cloud Platform**. A GCP possui um sistema integrado de monitorização que nos permite observar a variação de dadas métricas durante a execução da aplicação **Laravel.io**.

A plataforma Google Cloud fornece métricas, eventos e metadados apresentados através da colheita de informação dos componentes envolvidos. Identifica problemas e descobre padrões, ajudando a avaliar a experiência que é possível fornecer a um utilizador e a facilitar a análise da qualidade da implementação.

Para a nossa solução, optamos por implementar um *dashboard*, com gráficos lineares e alguns valores numéricos. Estes permitem visualizar como varia a percentagem de CPU e a memória utilizada pelos nós criados, o número de *Pods* criados e o *throughput*. Podemos também visualizar como uso do CPU e da memória para cada *Container* (ex: *Laravel.io* e *MySQL*). De seguida, apresentamos todos os valores numéricos que conseguimos obter através do dashboard implementado :

1. Percentagem de Utilização do CPU
2. Memória Utilizada
3. Tempo de Uso do CPU
4. Tempo Requerido pelo CPU ao *Container* do *MySQL*
5. Tempo Requerido pelo CPU ao *Container* do *Laravel.io*
6. Percentagem Média de Utilização do CPU por Nó
7. Média da Memória Utilizada por Nó
8. Throughput Médio
9. Memória Usada/Limite do Container do *MySQL*

Optamos por criar este *dashboard*, uma vez que consideramos que os restantes *dashboards* disponibilizados pelo GCP eram relativamente confusos, de difícil compreensão e visualização e que geravam a necessidade de utilizar filtros. Assim, ao automatizarmos a criação do *dashboard* escolhido, podemos mais facilmente observar as métricas que consideramos pertinentes e evitamos o processo de aplicar filtros aos *dashboards* já existentes.

Assim, a monitorização permitiu-nos compreender como o programa iria utilizar os recursos que lhe estavam disponíveis e visualizar as anomalias que poderiam ocorrer durante a execução, bem como, identificar possíveis gargalos de desempenho associados a diferentes componentes. Com estas métricas escolhidas, desenvolvemos um **Ansible Playbook** para o *role* de monitorização. Este *playbook* permitiu-nos criar os *dashboards* com as métricas escolhidas de forma automática, recorrendo ao módulo *command*. Os *dashboards* que definimos serão criados após a inicialização da aplicação **Laravel.io** e poderão ser catalogados na categoria *Custom* e consultados na janela de monitorização na página *browser* do GCP.

Avaliação Experimental

Com a escolha das ferramentas e métricas utilizadas para a monitorização da aplicação **Laravel.io** efetuada, foram-nos propostas algumas questões relacionadas com o funcionamento da aplicação, sendo estas as seguintes:

1. Para um número crescente de clientes, que componentes da aplicação poderão constituir um gargalo de desempenho?
2. Qual o desempenho da aplicação perante diferentes números de clientes e cargas de trabalho?
3. Que componentes da aplicação poderão constituir um ponto único de falha?
4. Que otimizações de distribuição/replicação de carga podem ser aplicadas à instalação base?
5. Qual o impacto das otimizações propostas no desempenho e/ou resiliência da aplicação?

Para nos ajudar a responder a estas questões, decidimos efetuar alguns testes e a sua respetiva monitorização de modo a obter resultados concretos sobre a resposta a estas questões. Claro que, temos uma ideia das respostas, pelo que antes de execução dos testes, assumimos que:

1. Para um número crescente de clientes, será de esperar que a Base de Dados seja a componente cujo desempenho será mais crítico, uma vez que deverá executar várias *queries*, enquanto que a aplicação **Laravel.io** será responsável essencialmente por redirecionar pedidos dos clientes para a Base de Dados e devolver a resposta, executando desta forma uma carga computacional menor.
2. Será também expectável que, para um número elevado de clientes a percentagem de CPU e de memória utilizada ira aumentar significativamente, bem como, as cargas de trabalho em ambos os componentes.
3. Em relação a pontos únicos de falha, pensamos que tanto o **Laravel.io** como a Base de Dados serão componentes críticos, na medida em que se qualquer um deles falhar, o serviço deixa de funcionar e estar disponível.
4. As otimizações/distribuição de carga que podem ser aplicadas à instalação base da aplicação prendem-se com possuir pelo menos duas instâncias de cada componente de forma a que se um destes componentes falhar, o serviço poderá ser mantido pelo componente restante, até que o outro recupere, servindo também como forma de divisão de carga.
5. Esperamos que as otimizações propostas no ponto acima aumentem o desempenho e a resiliência/elasticidade da aplicação.

Para, de facto, podermos responder concretamente a estas questões e com testes que o provem, decidimos implementar um mecanismo de *benchmarking* utilizando a ferramenta **Jmeter** e o monitoring disponibilizado pelo GCP. De salientar que também esta ferramenta é capaz de ser iniciada e de executar testes automaticamente a partir do *playbook*.

Na nossa solução, utilizamos a ferramenta Jmeter para desenvolver um conjunto de três diferentes *workflows* que permitem simular diferentes comportamentos dos clientes aquando do seu acesso a aplicação **Laravel.io**. Estes foram criados com o objetivo de simular uma interação realista entre o cliente e a aplicação. Como vamos ver de seguida, temos um *workflow* básico e temos dois *workflows* mais complexos, pois achamos que deveríamos testar a aplicação para situações que pudessem ocorrer no mundo real, coisa que um *workflow* que apenas acede a página inicial não satisfaz. De seguida, apresentam-se os *workflows*.

1. **Simples** - Acesso à página Login.
2. **Normal** - Login, acessar o Fórum, Logout.
3. **Complexo** - Login, acessar Fórum, criar Thread, Logout.

Para o workflow simples, utilizamos a ferramenta Jmeter para desenvolver um conjunto sintético de *requests HTTP* para a página de *login* do **Laravel.io**. Começamos por definir uma *Thread Group* que representa os *benchmark clients* que efetuam *requests HTTP*. Após a definição de *benchmark clients*, implementamos um *timer* aos *requests* a ser feitos à página do **Laravel.io** pretendida. Com isto, e variando o número de *threads* pertencentes à *Thread Group* (número de clientes), pretendemos visualizar através das ferramentas de monitorização definidas anteriormente o comportamento e a variação das métricas monitorizadas no *cluster*.

Para o *workflow* normal, efetuamos o mesmo procedimento que para o *workflow* simples, mas desta vez adicionando mais quatro *requests HTTP*, que são respetivamente :

1. *Request* para a página de Post Login
2. *Request* para a página do Utilizador
3. *Request* para a página do Fórum
4. *Request* para a página de Logout

Com isto, e variando o número de *threads* da *Thread Group* (nº de clientes) pretendemos visualizar a variação das métricas monitorizadas no *cluster*, com especial atenção no *throughput* visto que este *workflow* possui uma carga de trabalho superior ao anterior.

Finalmente, para o *workflow* complexo, efetuamos o mesmo procedimento que para o *workflow* normal, adicionando mais um *request HTTP* associado à criação de uma *thread* no fórum.

Com isto, e variando o número de *threads* da *Thread Group* (nº de clientes) pretendemos visualizar a variação das métricas monitorizadas no *cluster*, com especial atenção no *throughput* visto que este *Workflow* possui uma carga de trabalho superior ao anterior.

Importa salientar que os *workflows* acarretam um aumento em termos de carga computacional, assim como o número de operações. Efetuamos diversos testes, alterando tanto o número de clientes como o tipo de *workflow*, e analisamos a percentagem de CPU e de memória utilizada, assim como o *throughput*. As tabelas seguintes apresentam os valores obtidos nestes testes:

Nº de Clientes	Throughput (B/s)	CPU (%)
0	0.0	59.5
1	42.5	63.7
2	78.22	69.4
4	97.28	71.7
8	112.1	88
16	119.2	102

Tabela 1: Tabela que relaciona o Nº de Clientes com o Throughput Médio, para o Workflow Simples

Nº de Clientes	Throughput (B/s)	CPU (%)
0	0.0	59.5
1	77.4	75.6
2	82.2	80.2
4	86.8	88.4
8	97.3	95.6
16	100	112

Tabela 2: Tabela que relaciona o Nº de Clientes com o Throughput Médio, para o Workflow Normal

Nº de Clientes	Throughput (B/s)	CPU (%)
0	0.0	59.5
1	97.3	84.2
2	98.3	85.1
4	98.4	92.7
8	112.2	105
16	117.1	137

Tabela 3: Tabela que relaciona o Nº de Clientes com o Throughput Médio, para o Workflow Complexo

Como podemos observar pelos resultados obtidos, concluímos que a nossa aplicação não consegue suportar mais que oito clientes que apenas acedam a página de *login* e mais de quatro clientes que efetuem *login* na aplicação. Isto deve-se ao facto de que o CPU do *pod* aplicacional não consegue lidar com muitos pedidos de vários clientes em simultâneo, visto que a sua percentagem de uso ultrapassa os 100% na maior parte dos casos em que quatro a oito clientes estão a utilizar a aplicação.

De realçar também que, embora o *throughput* aumente conforme o aumento do número de clientes, verifica-se que este aumento é cada vez menor quanto maior for o número de clientes, ou seja, a aplicação não consegue aumentar a quantidade de trabalho efetuada de forma proporcional ao aumento do número de clientes, pois não dispõem dos recursos necessários para tal. Um destes recursos é a memória, que não foi incluída nas tabelas de resultados pois esta mantinha-se semelhante na grande maioria dos casos, o que nos leva a concluir que a memória disponível constitui um gargalo de desempenho, pois não aumenta conforme o aumento do número de clientes. Como todos os pedidos dos clientes vão acabar no único *pod* de Base de Dados disponível, esta não possui capacidade para aumentar a memória disponível e satisfazê-los a todos em simultâneo, daí o *throughput* não aumentar de forma significativa conforme o aumento do número de clientes.

Regressando agora às questões que nos foram colocadas, podemos concluir que :

1. Os componentes que constituem um gargalo de desempenho são os *pods* tanto aplicacional como da Base de Dados.
2. A aplicação diminui a sua *performance* e capacidade de resposta a pedidos conforme o aumento do número de clientes e o aumento da carga de trabalho.
3. Ambos os *pods* constituem um ponto único de falha, pois se algum destes falhar, todo o serviço fica comprometido.
4. As otimizações que podem ser realizadas na instalação base da aplicação incluem a replicação e sincronização do *pod* da Base de Dados, bem como a replicação do *pod* aplicacional.
5. Para as otimizações mencionadas acima, é expetável que ambas aumentem a resiliência e o desempenho da aplicação.

Escalabilidade e Resiliência

Esta tarefa consiste em conseguir replicar e melhorar o desempenho de um dos componentes da aplicação **Laravel.io**. Assim, decidimos focar-nos apenas no **Servidor Aplicacional**.

Como observamos anteriormente, o nosso sistema foi implementado seguindo uma arquitetura **Multitier**. Assim, a nossa solução deverá ser capaz de apresentar escalabilidade e resiliência.

Como passo seguinte da nossa solução, atribuímos ao nosso *deployment* a capacidade de criar réplicas para o *pod* do Servidor Aplicacional da aplicação **Laravel.io** tendo em vista a taxa de utilização de CPU do mesmo, bem como a média. Deste modo, pretendíamos fornecer maior poder computacional e maior resiliência à nossa aplicação, dado que existem mais *pods* disponíveis e caso um *pod* fique indisponível continuávamos a ter pontos de acesso à aplicação.

Utilizando o `HorizontalPodAutoScaler`, quando a aplicação **Laravel.io** se encontrar sobrecarregada, isto é, com uma taxa de utilização de CPU superior a 80%, será criada mais uma réplica do Servidor Aplicacional do **Laravel.io**. Este componente é iniciado com um *pod* e serão criadas réplicas da forma explicada anteriormente, sendo o número máximo criado é três. Na nossa solução não é possível criar mais *pods*, uma vez que o número de nós utilizados fornece CPU que permite a criação de apenas três réplicas, tendo em conta o CPU atribuído a cada *pod*.

Como mencionámos anteriormente, quando implementamos o **Laravel.io** tornámos a aplicação acessível através de um *browser* utilizando um *LoadBalancer*. Graças a este, é expectável que a "carga de trabalho" seja distribuída pelas diferentes réplicas do Servidor Aplicacional do **Laravel.io**.

Para analisarmos o funcionamento do `Horizontal Pod Autoscaler`, utilizámos a avaliação experimental definida na etapa anterior. Definimos que o *JMeter* iria executar dezasseis *workflows* complexos a cada segundo. Quando estes pedidos foram feitos, o *container* reagiu como na imagem apresentada de seguida:

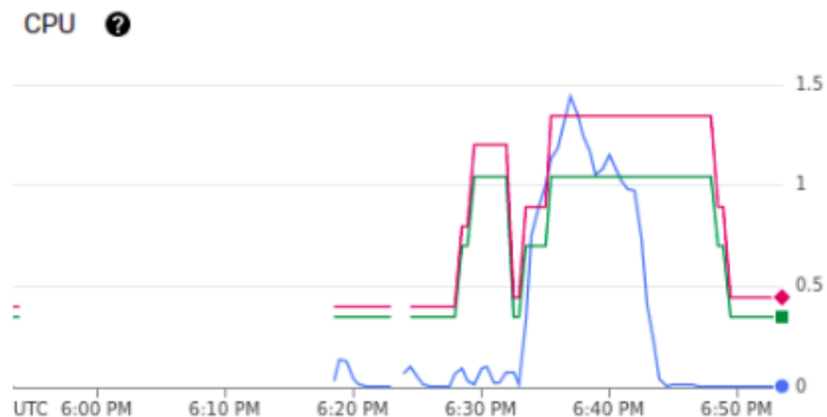


Figura 2: Variação da utilização do CPU

A linha azul representa o CPU utilizado para responder aos pedidos recebidos, a linha verde o CPU *request* para o *container*, e a linha cor de rosa o CPU *limit* para o *container*.

Como podemos observar quando o CPU utilizado aumenta para além do *request*, a quantidade de CPU *request* e CPU *limit* aumenta. Este aumento deve-se à inicialização de um novo *pod* para este *container*. Como podemos observar, posteriormente, quando os *pods* não forem necessários, estes ser irão ser removidos.

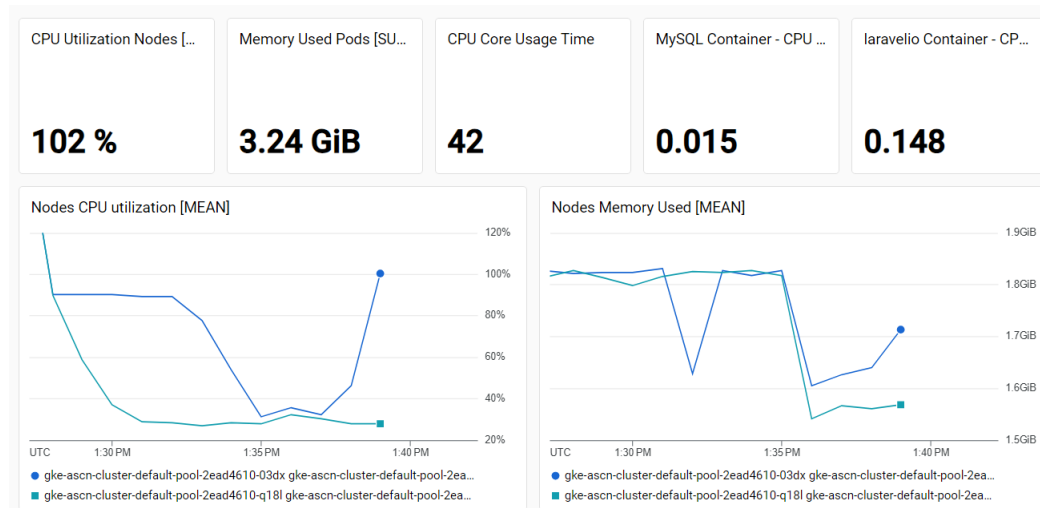


Figura 3: Variação da percentagem de CPU e da Memória com Autoscaler

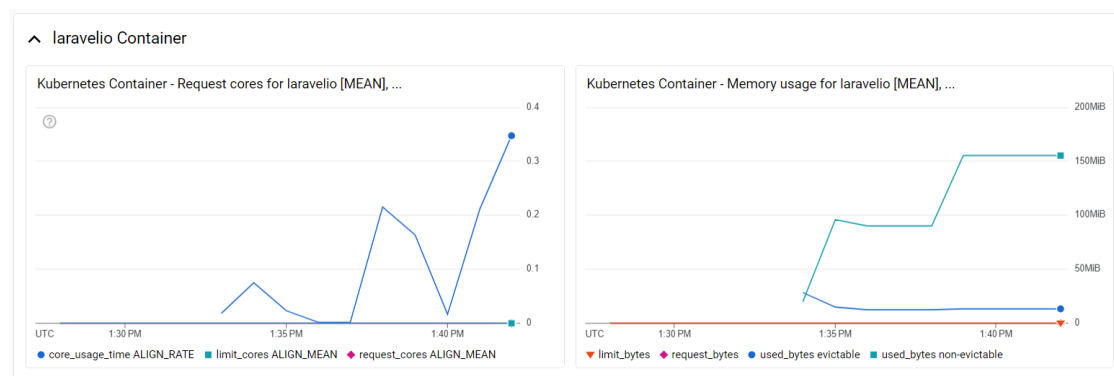


Figura 4: Variação da Requisição de Pods e Utilização de memória com Autoscaler

Como podemos observar pelos gráficos apresentados, verifica-se que quando é atingida uma percentagem de utilização de CPU superior a 80%, é requisitado um pod aplicacional para ser possível continuar a servir os pedidos dos clientes de forma eficaz, o que confere elasticidade e resiliência a aplicação. Além disso, é de salientar que a CPU(%), para o mesmo teste de 16 clientes em simultâneo num workflow complexo, apresentou uma descida de 137% para 102%.

Conclusão e Trabalho Futuro

Em conclusão, consideramos que os requisitos foram integralmente atendidos e que o deployment da aplicação **Laravel.io** está plenamente funcional, demonstrando escalabilidade e resiliência. Desta forma, atribuímos um balanço positivo ao trabalho desenvolvido, pois proporcionou a oportunidade de aplicar e consolidar os conceitos abordados, tanto a nível teórico como prático, ao longo do semestre.

A nossa implementação recorre a funcionalidades do **Google Cloud**, como, por exemplo, o *Google Kubernetes Engine* e *Ansible*, que ajudaram a automatizar o *deployment*, escalabilidade e gestão da aplicação **Laravel.io**, balanceando também a carga através de um *LoadBalancer*.

Numa análise final, gostaríamos de salientar que, infelizmente, constatamos que a nossa solução não apresenta um desempenho ideal. Apesar da implementação de um **Horizontal Autoscaler** e do sucesso do *deployment* automático ao lidar com o aumento do número de clientes e da carga total de trabalho, com o número máximo de nodos que utilizamos (três), a solução não consegue servir muito mais do que dezasseis clientes em simultâneo, num cenário real onde seria expectável que estes utilizassem a grande maioria das funcionalidades da aplicação. Isso ocorre devido à necessidade de um maior número de pods para manter um uso de CPU razoável.

Outra razão pela qual a nossa aplicação não consegue suportar um número mais elevado de clientes prende-se com o facto de possuímos apenas um pod de MySQL, ou seja, possui apenas um pod de base de dados o que implica que todos os pedidos efetuados nos vários pods aplicacionais sejam enviados para um único pod de base de dados. Como trabalho futuro e para aumentar o desempenho da aplicação seria fundamental ocorrer replicação e sincronia de pods da base de dados.

Gostaríamos também de realçar que, num contexto de trabalho futuro, seria fundamental a utilização da ferramenta **Ansible Vault**, pois permitiria que informação sensível como palavras passe ou o ficheiro JSON da service-account ficassem encriptados e, desta forma, mais protegidos.